
Chapter 4 | 模型训练与量化导出

- 本章目标：给 NPU 提供权重和验证数据。
- 内容：tiny CNN、MNIST、后训练量化、导出格式。
- 产出：npu_mnist.py + npu_data/ 目录。

Chapter 4 | 为什么先做分类

- 分类模型只有 conv + pool + FC。
- 没有 YOLO 的检测头、NMS、多尺度。
- MNIST 小、训练快、验证直观。
- 跑通后可以替换成更大的数据集/模型。

Chapter 4 | 模型结构

- Conv(1- $\rightarrow$ 8, 3x3) + ReLU + MaxPool - $\rightarrow$ 14x14x8。
- Conv(8- $\rightarrow$ 16, 3x3) + ReLU + MaxPool - $\rightarrow$ 7x7x16。
- Flatten(784) - $\rightarrow$ FC(64) - $\rightarrow$ ReLU - $\rightarrow$ FC(10)。
- 参数约 5 万，int8 权重约 50KB。

Chapter 4 | 训练要点

- 固定随机种子，保证可复现。
- MNIST 归一化到 $[0,1]$ 或 $[-1,1]$ 。
- Adam + 少量 epoch 即可收敛。
- 目标：float 准确率 $> 97\%$ ，int8 后 $> 95\%$ 。

Chapter 4 | 为什么要量化

- 硬件乘法器是 $\text{int8} \times \text{int8}$ 。
- float32 权重会占 4 倍存储。
- int8 推理是嵌入式 NPU 的事实标准。
- 量化误差可以通过 scale 逐层校正。

Chapter 4 | 后训练量化

- 用校准集统计每层激活的 min/max。
- $\text{scale} = (\text{max} - \text{min}) / 255$, zero_point 可省略（对称量化）。
- $q = \text{round}(\text{clamp}(x / \text{scale}, -128, 127))$ 。
- 累加结果反量化： $\text{out_fp} = \text{out_int} * \text{scale_act} * \text{scale_w}$ 。

Chapter 4 | BN 折叠

- BatchNorm 在推理时可以并入前一层卷积。
- $w_fold = w * \gamma / \sqrt{\text{var} + \epsilon}$ 。
- $b_fold = (b - \text{mean}) * \gamma / \sqrt{\text{var} + \epsilon} + \beta$ 。
- 这样 RTL 不需要单独实现 BN。

Chapter 4 | 导出格式

- `weights.hex` : 逐层权重, 行主序, 每行一个 int8 补码十六进制。
- `image.hex` : 一张测试图片。
- `golden.hex` : 每层 int8 输出 (或最终 logits) 。
- `scales.json` : 每层 `scale` / `zero_point` / 布局信息。

Chapter 4 | Python int8 模拟器

- 用 PyTorch 张量实现量化 forward。
- 每层输出量化回 int8，与 RTL 行为一致。
- 这就是 RTL testbench 的 golden。
- 先把 Python 模拟器调对，再碰硬件。

Chapter 4 | 常见坑

- round 规则不一致（Python 银行家舍入 vs 四舍五入）。
- scale 用 float 计算但导出时精度丢失。
- ReLU 之后负半轴被截断，min 不能直接用负值。
- 权重/激活布局与 RTL 地址映射不一致。

Chapter 4 | 本章任务

- 训练 tiny CNN 并量化。
- 实现导出脚本与 Python int8 模拟器。
- 生成 npu_data/ 全部文件。
- 提交量化精度报告。

Chapter 4 | 总结

- 模型结构要和硬件能力对齐：小、int8、无复杂算子。
- 量化不是精度损失的结束，而是工程取舍的开始。
- 导出格式就是硬件和软件之间的接口。
- 下一章：把 NPU 挂回 Hack Computer，跑端到端推理。